

Efficient Merkle-Tree Consistent Accumulator

Anna Mendonca^{†*}, Hudson Shi[†], Triet (PJ) Huynh[†], Ivan Pryvalov^{‡¶}, Amir Herzberg[†]

[†]University of Connecticut

[‡]Brandenburg University of Technology

[¶]University of Luxembourg

Abstract—A *consistent accumulator* computes a digest for a dynamically-growing set of elements, with a *proof of consistency* of the new digest with the previous digests. Consistent accumulators are in wide use, in particular, by Certificate Transparency (CT), which is part of the Web PKI, and in blockchains.

We present a significantly more efficient design for a consistent accumulator. Our design is compatible with the CT specifications; similarly to the widely-used, open-source CT implementation, it uses a Merkle tree, but much more efficiently. We provide open source implementation, security analysis and experimental evaluation showing the performance improvements.

Index Terms—Merkle tree, Cryptographic accumulators, Certificate Transparency, Proof-of-inclusion, Keyed accumulators

I. INTRODUCTION

Merkle trees and other cryptographic accumulators are widely used to accumulate and ensure integrity of large collections of data. A short *digest* authenticates an entire dataset and a compact *Proof-of-Inclusion* (PoI), also called a membership witness, for each data element, to allow verification that this data element belongs to the collection. Accumulators are crucial to append-only logs, blockchains, and any storage services that require a tamper-evident history with efficient verification.

In many real-world deployments, this collection of data elements grows over time. For example, in Certificate Transparency (CT) [1], a log operator must continuously add new certificate entries, and clients and monitors are able to track the log’s growth, verifying *consistency* between the old and new digests. It is also desirable to be able to efficiently update previously issued Proofs-of-Inclusions (PoIs).

Another example is blockchains, where Merkle trees allow for short proofs of inclusion of a particular block in the global state. While such compact proofs enable light (stateless) clients to verify the transactions without knowing the full state, it is also desired to efficiently maintain the consistency of such proofs as the blockchain grows.

In this work, we present the Efficient Merkle-tree Consistent Accumulator ($\mathcal{CMT}^h$), an efficient construction of a consistent accumulator, based on Merkle trees. $\mathcal{CMT}^h$ can be keyless, using a keyless hash h , or keyed, using a keyed hash function h ; when using a key, the key is chosen randomly and made public (non-secret). $\mathcal{CMT}^h$ is significantly more efficient than current constructions for consistent accumulators, such as the

transparency-dev repository used by CT implementations [2], while being compatible with the CT specifications.

$\mathcal{CMT}^h$ is also compatible with other existing Merkle-tree implementations of an accumulator. It produces the same digests and the same PoIs for the same sequences of elements, so existing verifiers do not need to change their code or trust model. This design choice keeps deployment costs low, and makes the benefits directly applicable to systems already built around Merkle trees.

To achieve this high efficiency, $\mathcal{CMT}^h$ implements several functionalities and satisfies requirements that are beyond the classical definitions of cryptographic accumulators. Therefore, we define *consistent accumulator schemes* and their requirements, including definitions for correctness requirements and for security requirements, and unforgeability requirements. This allows us to prove that $\mathcal{CMT}^h$ satisfies these requirements. Our definition of consistent accumulators allows us to expose the efficiency gain of our construction. In particular, unlike previous definitions where each new element is accumulated individually, in our definition, every accumulation operation receives a sequence of elements, allowing better efficiency than adding one element at a time. Also, accumulation operations output not only proofs-of-consistency (PoCs) but also update values, which allow verifiers to efficiently update proofs-of-inclusion (PoIs), and a separate function allows efficient joining of these update values. The accumulator also supports efficient retrieval of PoCs that prove consistency between any pair of accumulation operations. These definitions of a consistent accumulator and its requirements allow the creation of other efficient designs, e.g., designs not related to the Merkle tree.

Our design is compatible with real-world systems, in particular and primarily, Certificate Transparency (CT) [3], [1], where Merkle trees are used to audit and verify large append-only logs. In such settings, clients must continuously validate updated digests and reuse existing proofs efficiently. While CT is widely adopted, $\mathcal{CMT}^h$ can be used for many other applications that require efficient accumulation with proofs of consistency, e.g., blockchains.

CONTRIBUTIONS:

- 1) *Efficient Merkle-tree Consistent Accumulator*: We present $\mathcal{CMT}^h$, an efficient Merkle-tree Consistent Accumulator, constructed using a hash function h . The design is compatible with the ‘classical’ (non-consistent and/or non-optimized) Merkle tree implementations, making it easy to use in existing systems.
- 2) We provide open-source implementation of $\mathcal{CMT}^h$ in

This is the full version of the paper.

* Corresponding author: anna.mendonca@uconn.edu

Go [4], and show its significant performance advantages in computation time compared to Google’s open-source transparency.dev implementation [2], also implemented in Go.

- 3) *Definitions for Consistent Accumulators:* We define *consistent accumulator schemes* and *extended consistent accumulator schemes*, including security properties tailored to the dynamic setting. In our definitions, a consistent accumulator supports adding a *batch* of elements in one accumulation event, returns PoIs for the newly added elements, and returns a compact update value and proof of consistency for moving from the old digest to the new one. This is important in practice, since systems such as CT and blockchains naturally add many elements at once rather than one by one, and treating a batch as a first-class operation avoids unnecessary recomputation.
- 4) *Security Proofs:* We provide proofs showing that our design meets the security requirements of consistent accumulators.

A. High Level Overview of $\mathcal{CMT}^h$

Our design is centered around the *dyadic cover* of the Merkle tree, i.e. the minimum set of full subtrees that completely cover the full state of the log. The roots of these subtrees are what $\mathcal{CMT}^h$ stores as its state digests. As will be discussed in a later paragraph, the proofs of consistency generated by $\mathcal{CMT}^h$ are also based on the dyadic cover.

A crucial part of $\mathcal{CMT}^h$ ’s space savings comes from the fact its state digest set, the dyadic cover, grows $\log(n)$ with respect to the size of the tree. This fact is apparent in Fig. 1, where the dyadic cover is shown by the brown digests, with each such digest being the root of a full subtree. In the worst case, when the tree size is $n = 2^k - 1$, the size of this dyadic cover, and therefore the state digests, is $\log(n + 1)$.

$\mathcal{CMT}^h$ returns the proof of consistency (PoC), as well as an *update*, for the input sequence batch. To support maximally-compact dyadic covers for the PoC and update, we add an extended Definition 10 to support retrieving the most compact PoC and update between any two batch boundaries in the state history. The most compact PoC or update is the dyadic cover of the leaf difference, the leaf nodes that are present in the final state but not in the initial. This is useful as using our main (unextended) definition set, the PoC or update across multiple accumulations would include redundant digests because the digests are directly taken from the single batch accumulations.

$\mathcal{CMT}^h$ optimizes for space efficiency by allowing additions of batches instead of singular elements. This means only supporting PoCs between these batches, specifically, the states produced by a call to Accum, rather than for every individual element addition. This choice limits the granularity of verifiable states, but it enables the log to remain performant and scalable under high-throughput conditions.

B. Related Work

Cryptographic accumulators [5], [6] accumulate multiple data elements into a compact digest and allow compact *proofs*

of inclusion (PoIs). Accumulators are widely used in practice, most commonly, using designs based on Merkle trees [5], e.g., used in Certificate Transparency (CT) [3], [1], [7]. CT, and other applications, allow multiple accumulation events, using *proofs of consistency (PoCs)* between two digests. Informally, proofs of consistency guarantee that data elements accumulated in the older digest are also present in the newer digest. However, in spite of the extensive use of CT and other Merkle-tree system supporting multiple accumulation events, we are not aware of efforts to improve its performance, or, in fact, of analysis showing its security properties.

There are many works extending accumulators to allow for multiple accumulation events [8], [9], [10], [11], [12], [13], [14], [15]. A smaller number of works also ensure *proofs of consistency (PoCs)*. Crosby and Wallach [16] use Merkle trees as a basis for constructing *tamper evident logging*. Dowling et al. [17] use Merkle trees in append-only *secure logging schemes*, while Kales et al. [18] extend this setting for privacy-preserving retrieval. Tomescu et al. [19] introduce *append-only authenticated dictionaries* and use them for building transparency logs, allowing key-search.

CONIKS [20] uses prefix Merkle trees to implement transparent name-to-key bindings, which facilitate a key-search and both proofs of inclusion and non-inclusion. Extensions of CT to support *key transparency* have recently received attention [21], [22], [23]; such protocols allow individual updates by replacing previously accumulated elements. While offering additional functionality and supporting different system models, these data structures require at least a linear state.

We believe no prior Merkle-tree design allowed multiple accumulations with a *compact state*, or provided (efficient) *proofs of consistency (PoCs)*. The asynchronous accumulator by Reyzin and Yakoubov [24], where the proof of inclusion can be valid after a certain number of modifications to the accumulator (update frequency is logarithmic in the number of elements), has a compact state, however no proofs of consistency. The closest work to ours is that of Crosby and Wallach [16]. In both [16] and our work, standard Merkle trees are used as a basis and essential nodes are used for proving updated membership and consistency. However, our design is much more efficient in storage and computations (e.g., $O(\log n)$ vs. $O(n)$), in addition, it can return an update value between any two states, thereby reducing the overall verification complexity for users. We refer to Table I for a comparison of our work to related work.

Our technique is similar to *Merkle Mountain Ranges* [26] used by Reyzin and Yakoubov [24] to describe the accumulator by splitting a single tree into multiple full binary subtrees, whose size is a power of two. When adding a new element, two full subtrees of equal size are merged into a new subtree by connecting their root nodes. While we use the roots of the subtrees to describe the state of the accumulator, our design requires only a single root node as digest, allowing it to be compatible with other Merkle tree designs. The idea of caching the roots of full subtrees to avoid unnecessary recomputations was suggested in [16]; such nodes are called “frozen”. Our consistency proofs between two accumulator states follow [16],

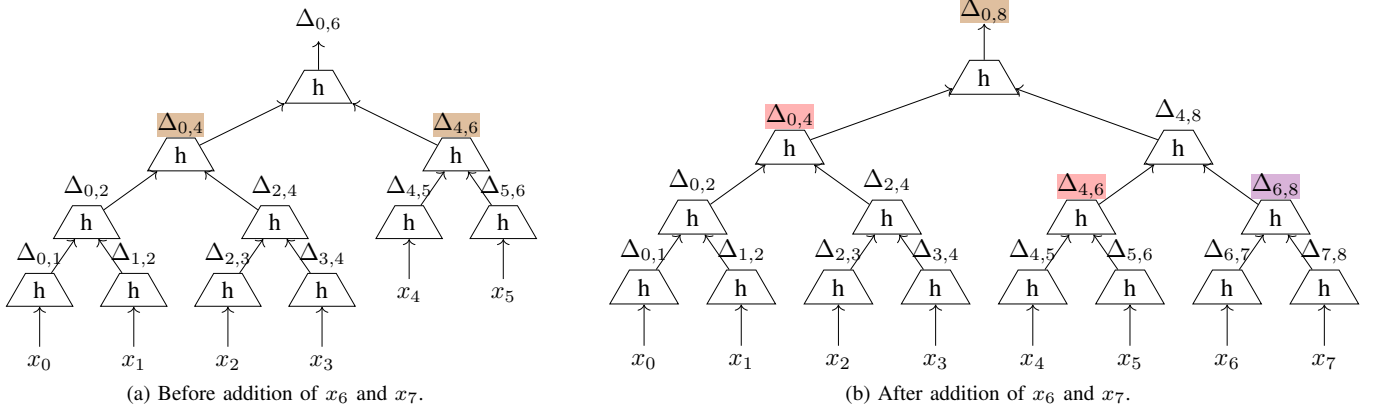


Fig. 1: An Efficient Merkle-tree Consistent Accumulator ($\mathcal{CMT}^h$) accumulator update adding elements x_6 and x_7 . The state's digest set transitions from $\{\Delta_{0,4}, \Delta_{4,6}\}$ (brown in Fig. 1a) to $\{\Delta_{0,8}\}$ (Fig. 1b). The old state digests are shown in red, and the update value u (which also serves as the *PoC*) is $\Delta_{6,8}$ (violet).

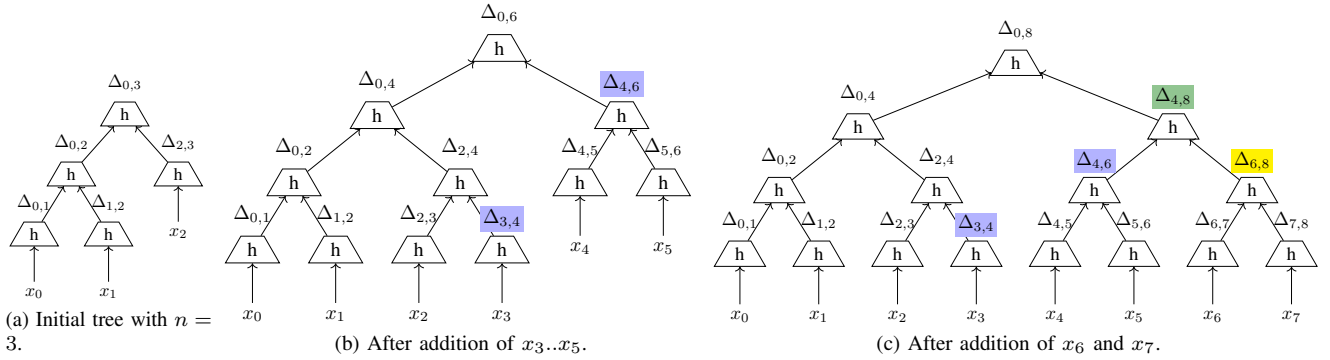


Fig. 2: An Efficient Merkle-tree Consistent Accumulator ($\mathcal{CMT}^h$) update adding a batch of 3 elements x_3, x_4, x_5 , followed by another batch of 2 elements x_6, x_7 . For 2b and 2c, their corresponding update or proof-of-consistency nodes from the previous state are highlighted in blue ($\{\Delta_{3,4}, \Delta_{4,6}\}$) and yellow ($\{\Delta_{6,8}\}$), respectively. These nodes are added to $s.UM$, as well as $\Delta_{4,8}$ (shown in green) to hold the invariant that a parent must exist in $s.UM$ if both children also are present. As a result, a client can request any of the three possible PoCs $\{(a) \rightarrow (b), (b) \rightarrow (c), (a) \rightarrow (c)\}$. The batch addition $(a) \rightarrow (b)$ corresponds to the PoC digests $\{\Delta_{3,4}, \Delta_{4,6}\}$ (blue, blue); $(a) \rightarrow (c)$ corresponds to $\{\Delta_{3,4}, \Delta_{4,8}\}$ (blue, green); $(b) \rightarrow (c)$ corresponds to $\{\Delta_{6,8}\}$ (yellow).

Work	State	PoI	PoC	Accum (runtime, digest)	Update PoI	Trustless setup
[16], [17], [25]	n	$\log n$	$\log n$	$\log n$, digest 1	<i>not supported</i>	Yes
[19]	λn	$\log n$	$\log n$	$\lambda \log^3 n$, digest 1	<i>not supported</i>	No
$\mathcal{CMT}^h$	$\log n$	$\log n$	$\log n$	$\log n$, digest 1	$m + \log n$	Yes

TABLE I: Comparison of our Merkle-tree consistent accumulator $\mathcal{CMT}^h$ with related publications; better values in bold. All numbers are in $O(\cdot)$ notation; n is the number of elements in tree, λ the security parameter. *Accum* numbers are given for accumulating a single element, which is supported by all constructions. All publications feature a Proof-of-Inclusion (PoI) and Proof-of-Consistency (PoC). *Update PoI* refers to updating an outdated Proof-of-Inclusion to be compatible with a newer tree having m more elements. All works use a Merkle tree-like design, except for [19], which is based on bilinear maps.

[17], adapted to require only *small state*.

The dynamic addition of *multiple* elements is known as *batched updates*, e.g., in the context of blockchains [27], [28]. Here, the batched proof aggregates multiple individual proofs of inclusion. Such aggregated proofs are more compact than individual ones, due to the tree structure and the fact that neighboring elements may share intermediate nodes [27]. In our design, we mainly focus on reducing the *server* state when adding multiple elements at once and techniques such as [27] are orthogonal to our approach. In the context of

definitions for *accumulators* and their security properties, the functionality of adding multiple elements is defined for a single element [8], [24], [13], [15], [18], thus we start by formalizing a system which allows us to define, instantiate and analyze a consistent accumulator that supports efficient batched additions. We extend the definitions in [24], to create a unified formal model that allows for *batched* accumulation of new elements. In particular, this model suits real implemented systems, for example, in both CT logs and blockchains.

Related to accumulators notions are polynomial commit-

ments [29] and vector commitments (VCs) [30]. In the former notion, a single value represents a commitment to a polynomial and allows proofs of correct polynomial evaluations. The latter is a fixed-length primitive, a more generic than Merkle Trees, allowing constant size proofs of membership and efficient update of committed elements. Aggregate proofs are possible as well (e.g., [31]). In our work, we diverge from VCs by considering the append-only data structure with the consistency property, ensuring the correct accumulation of new elements.

Specific to the blockchain applications, Coded Merkle Trees [32] address the data availability problem. Their essential part is the ability to generate the so-called fraud proofs w.r.t. the ledger state and the semantic correctness of block transactions. This is beyond the scope of our paper.

II. CONSISTENT ACCUMULATORS

Notations. We use $\langle X_i \rangle_{i=1}^l$ to denote a finite sequence, each of whose elements X_i is itself a sequence of binary strings, i.e., $X_i \equiv \langle x_j \rangle_{j=0}^{m-1}$ where $x_j \in \{0, 1\}^*$. The union operation, when applied to two or more sequences, outputs the (unordered) set containing the elements that appear in any of the sequences, i.e.,

$$\begin{aligned} \bigcup \langle X_i \rangle_{i=1}^l &\equiv X_1 \cup \dots \cup X_l \\ &\equiv \{x \in X_i \text{ for some } i \text{ s.t. } 1 \leq i \leq l\} \end{aligned}$$

To refer to a particular output parameter of a function of the accumulator, we append the parameter name to the function name, separated by a dot. For example, let algorithm Alg take as input par and output (a, b) , namely $(a, b) \leftarrow \text{Alg}(par)$. Then $\text{Alg}(par).a$ denotes the first element of the output pair, $\text{Alg}(par).b$ the second element.

Accumulators compute the digest of a set of elements; the digest can be used to ensure the integrity of the set, and the fact that a particular element was included in the set resulting in the given digest. We define *Consistent accumulators*, i.e., (stateful) accumulators that support multiple accumulation events, in each of which, a set of elements is added to the elements accumulated so far. Consistent accumulators provide *Proof of Consistency* (PoC), allowing validation that a new digest is the result of accumulating additional elements to the set of elements accumulated into a previous digest. This can be important for different applications; e.g., a specific consistent accumulator, based on the *Merkle Tree*, is used by the *Certificate Transparency* (CT) standards [25], [1].

To key or not to key. Applications such as CT usually use keyless accumulators. However, we present keyed definitions and constructions, using keyed hash functions and a random string as a key; this is since keyless accumulators, like keyless hash functions, cannot ensure collision resistance (under the standard security model). When using a keyless hash, our constructions are secure under the Random Oracle Model (ROM).

a) Why new definitions: In contrast to previous definitions where new elements had to be accumulated one at a time [8], [24], [13], [15], [18], the consistent accumulator accumulates,

each time, a whole set of strings. It also includes functions for updating PoIs (proofs of inclusion) and for generating and validating updates and PoCs (proofs of consistency) between the digests and PoIs produced in arbitrary accumulations. The support for accumulating sets of strings in each accumulation, and the new functions, allow the consistent accumulator to provide significantly more efficient accumulation service to applications.

To support accumulation of multiple sets of strings (in multiple accumulation events), the accumulation operations are done by a *stateful manager entity*. The verification operations are still stateless, and include verification algorithms such as VerPol, VerUpd, and VerPoC. The last two algorithms are used to check the validity of PoC and update values, respectively.

Definition 1 (Consistent accumulator schemes). A *consistent accumulator* $\alpha = (\text{Accum}, \text{VerPol}, \text{VerUpd}, \text{VerPoC}, \text{UpdPol})$ is a collection of PPT (Probabilistic Polynomial Time) algorithms, where, for every integer λ , key $k \in \{0, 1\}^\lambda$, binary strings $s, p, \Delta, \Delta', x, u$ and sequences of binary strings X, X' , we have $\alpha.\text{Accum}(X \| X', \emptyset) = \alpha.\text{Accum}(X', s)$ where $s = \alpha.\text{Accum}(X, \emptyset).s$, and:

- $\alpha_k.\text{Accum}(X, s) \rightarrow (s', \Delta, \text{PoI}, u, \text{PoC})$: given the current accumulator state¹ s and a sequence of strings $X = \langle x_i \rangle_{i=0}^{n-1} \in (\{0, 1\}^*)^n$, outputs an updated state s' , a digest $\Delta \in \{0, 1\}^\lambda \cup \{\perp\}$, a set of proofs-of-inclusion (PoIs) PoI , an update value u and a proof of consistency PoC .
- $\alpha_k.\text{VerPol}(\Delta, x, p) \rightarrow \{\text{TRUE or FALSE}\}$: given a digest Δ , an element x , and its corresponding proof-of-inclusion p , where $0 \leq i \leq |\text{PoI}|$, return TRUE if the proof is valid (i.e., x was included in the set represented by Δ), and FALSE otherwise.
- $\alpha_k.\text{VerUpd}(\Delta, u, \Delta') \rightarrow \{\text{TRUE or FALSE}\}$: given an initial digest Δ , an update value u , and a new digest Δ' , return TRUE if u is a valid update value from digest Δ' to Δ , and FALSE otherwise.
- $\alpha_k.\text{VerPoC}(\Delta, p, \Delta') \rightarrow \{\text{TRUE or FALSE}\}$: given an initial digest Δ , a proof of consistency p , and a new digest Δ' , return TRUE if Δ' is consistent with Δ , and FALSE otherwise.
- $\alpha_k.\text{UpdPol}(p, u) \rightarrow p'$: given a current proof-of-inclusion p for input string x , and an update value u , this algorithm returns an updated p' for x for use with the new digest Δ' .

We now present the requirements for consistent accumulators.

A. Correctness Requirements: definitions

Intuitively, a consistent accumulator ensures *PoI correctness* if for every string x accumulated into a digest Δ , there exist a corresponding Proof of Inclusion (PoI) such that $\text{VerPol}(\Delta, x, p) = \text{TRUE}$.

Definition 2 (PoI correctness). We say that a consistent (keyed) accumulator α scheme ensures *Proof-of-Inclusion* (PoI)

¹In the first call, we set the state s to the empty string ϵ .

Correctness if the following holds for every binary string key k , every binary string state s , and any finite sequence of binary string X :

$$\left(\begin{array}{l} \Delta \leftarrow \alpha_k.\text{Accum}(X, s).\Delta \wedge \\ \text{PoI} \leftarrow \alpha_k.\text{Accum}(X, s).\text{PoI} \end{array} \right) \Rightarrow \left(\begin{array}{l} (\forall x \in X)(\exists p \in \text{PoI}) \\ \alpha_k.\text{VerPol}(\Delta, x, p) = \text{TRUE} \end{array} \right)$$

Similarly, a consistent accumulator ensures *PoC and Update value correctness* if for any two digests Δ and Δ' , where Δ was the result of an accumulation that happened before Δ' , there must exist an update value u and a Proof of Consistency (PoC) such that $\text{VerUpd}(\Delta, u, \Delta') = \text{VerPoC}(\Delta, \text{PoC}, \Delta') = \text{TRUE}$.

Definition 3 (PoC and Update value correctness). We say that a consistent (keyed) accumulator α ensures PoC and Update value correctness if the following holds for every binary string key k , every initial binary string state s , and any finite sequence of binary strings: X and X' :

$$\left(\begin{array}{l} \Delta \leftarrow \alpha_k.\text{Accum}(X, s_0).\Delta \wedge \\ s_1 \leftarrow \alpha_k.\text{Accum}(X, s_0).s \wedge \\ \Delta' \leftarrow \alpha_k.\text{Accum}(X', s_1).\Delta \wedge \\ \text{PoC} \leftarrow \alpha_k.\text{Accum}(X', s_1).\text{PoC} \wedge \\ u \leftarrow \alpha_k.\text{Accum}(X', s_1).u \end{array} \right) \Rightarrow \left(\begin{array}{l} \alpha_k.\text{VerPoC}(\Delta, \text{PoC}, \Delta') = \text{TRUE} \wedge \\ \alpha_k.\text{VerUpd}(\Delta, u, \Delta') = \text{TRUE} \end{array} \right)$$

Let u be the result of GetU or Accum from digest Δ to Δ' . A consistent accumulator ensures *updating PoI correctness* if, when updating a PoI p for string x to p' using a valid update value u , the new PoI for x would be valid, i.e., $\text{VerPol}(\Delta', x, p') = \text{TRUE}$.

Definition 4 (Updating PoI Correctness). We say that a consistent (keyed) accumulator α ensures PoI Update Correctness if the following holds for every binary string key k , every initial binary string state s_0 , and any finite sequence of binary strings X and X' :

$$\left(\begin{array}{l} s_1 \leftarrow \alpha_k.\text{Accum}(X, s_0).s \wedge \\ \text{PoI} \leftarrow \alpha_k.\text{Accum}(X, s_0).\text{PoI} \wedge \\ \Delta' \leftarrow \alpha_k.\text{Accum}(X', s_1).\Delta \wedge \\ u' \leftarrow \alpha_k.\text{Accum}(X', s_1).u \end{array} \right) \Rightarrow \left(\begin{array}{l} (\forall x \in X)(\exists p \in \text{PoI}) \\ p' \leftarrow \alpha_k.\text{UpdPol}(p, u') \wedge \\ \alpha_k.\text{VerPol}(\Delta', x, p') = \text{TRUE} \end{array} \right)$$

B. Unforgeability requirements

Finally, we define three unforgeability requirements: *PoI unforgeability*, *update unforgeability* and *PoC unforgeability*. Note also that an accumulator is collision-resistant if for any PPT adversary the probability of finding two distinct sets of elements that produce the same digest is negligible. Proof of Inclusion (PoI) unforgeability guarantees collision resistance.

PoI unforgeability can be understood as the following. Consider any string x that was never the input to the accumulation

that generated Δ . Then the adversary cannot produce a proof of inclusion p that will verify as if x was one of the input strings for the accumulation operations that produces Δ , i.e. $\alpha_k.\text{VerPol}(\Delta, x, p) = \text{TRUE}$.

Definition 5 (PoI Unforgeability). Let α be a consistent accumulator scheme with key k . We say that α ensures *Consistent Accumulator Proof-of-Inclusion (PoI) Unforgeability* if for every PPT adversary $\mathcal{A}$, the following holds:

$$\Pr_{k \leftarrow \{0,1\}^\lambda} \left[\begin{array}{l} \alpha_k.\text{VerPol}(\Delta, x, p) = \text{TRUE} \wedge \\ \forall x \notin \bigcup(\xi), \text{ where } \\ \xi, x, p, \text{ and } \Delta \text{ were generated by:} \\ \left\{ \begin{array}{l} \xi \leftarrow \mathcal{A}(k); \\ \Delta \leftarrow \alpha_k(\xi, \epsilon).\Delta; \\ (x, p) \leftarrow \mathcal{A}(\xi, \alpha_k(\xi, \epsilon), k) \end{array} \right\} \end{array} \right] \in \text{Negl}(\lambda)$$

In the following definitions, we use the shorthand $\xi \equiv \langle X_j \rangle_j^l$ to denote a sequence of l sequences of binary strings, each $X_j \equiv \langle x_i \rangle_{i=0}^{m-1}$, as defined in Section II. In addition, the symbol ϵ is used to denote the empty string, which is also the initial state.

Update unforgeability requires that once a PoI is valid, applying any valid update value preserves validity; an attacker cannot use a valid update value to invalidate an existing PoI. If proof-of-inclusion p shows that string x was accumulated into digest Δ_0 , and u is a valid update from Δ_0 to Δ_1 , then the updated proof $p' \leftarrow \alpha_k.\text{UpdPol}(p, u)$ must also show that x was accumulated into Δ_1 .

Definition 6 (Update unforgeability). A (keyed) consistent accumulator scheme α ensures *Consistent Accumulator Update value u unforgeability* if for every PPT adversary $\mathcal{A}$, the following holds:

$$\Pr_{k \leftarrow \{0,1\}^\lambda} \left[\begin{array}{l} (\Delta_0, \Delta_1, u, x, p) \leftarrow \mathcal{A}(k) \text{ where:} \\ \alpha_k.\text{VerUpd}(\Delta_0, u, \Delta_1) = \text{TRUE} \wedge \\ \alpha_k.\text{VerPol}(\Delta_0, x, p) = \text{TRUE} \wedge \\ p' = \alpha_k.\text{UpdPol}(p, u) \wedge \\ \alpha_k.\text{VerPol}(\Delta_1, x, p') = \text{FALSE} \end{array} \right] \in \text{Negl}(\lambda)$$

Proof-of-Consistency (PoC) unforgeability requires that an adversary cannot produce a valid PoC showing that a new digest Digest_1 is consistent with a previous digest Digest_0 , given there is some string included in Digest_0 but not in Digest_1 . For a given state s and two sets $X_0 \not\subseteq X_1$, denote their corresponding digests as: $\Delta_0 \leftarrow \alpha_k.\text{Accum}(X_0, s).\Delta$ and $\Delta_1 \leftarrow \alpha_k.\text{Accum}(X_1, s).\Delta$. The consistent accumulator α ensures PoC unforgeability, if the adversary cannot find such s, X_0, X_1 and a valid *proof of consistency (PoC)* p from Δ_0 to Δ_1 , namely, $\alpha_k.\text{VerPoC}(\Delta_0, p, \Delta_1) = \text{TRUE}$.

Definition 7 (Proof-of-Consistency (PoC) unforgeability). We say that the (keyed) accumulator scheme α ensures Proof-of-Consistency (PoC) and update value Unforgeability if for

every PPT adversary $\mathcal{A}$ interacting with the accumulator α , the following holds:

$$\Pr_{k \leftarrow \{0,1\}^\lambda} \left[\begin{array}{l} (X, X', PoC, u) \leftarrow \mathcal{A}(k) \text{ where:} \\ (X \not\subseteq X') \wedge (|X| < |X'|) \wedge \\ \Delta \leftarrow \alpha_k.Accum(X, s). \Delta \wedge \\ \Delta' \leftarrow \alpha_k.Accum(X', s). \Delta \wedge \\ \alpha_k.VerPoC(\Delta, PoC, \Delta') = \text{TRUE} \end{array} \right] \in Negl(\lambda)$$

C. Collision resistance requirements

Similarly to the definitions of keyed hash functions in [33], we define two notions of collisions for (keyed) consistent accumulators: *target collision resistance (TCR)* and *any collision resistant* accumulators.

Let us begin with ACR accumulators, where the attacker is given a uniformly random key k and can then produce two arbitrary sequences of inputs to the accumulation operation, i.e., two sequences of sequences ξ, ξ' . The attacker ‘wins’ if the strings in ξ differ from the strings in ξ' , yet the digests of the ξ and ξ' are identical. The accumulator is ACR if this occurs with (at most) negligible probability as a function of λ , which is the key length $|k| = \lambda$.

Definition 8 (Consistent Accumulator Any-Collision-Resistant). We say that the (keyed) consistent accumulator scheme α is *Any-Collision-Resistant (ACR)* if for every efficient PPT algorithm $\mathcal{A}$ the following holds:

$$\varepsilon_{\alpha, \mathcal{A}}^{ACR}(\lambda) \equiv \Pr_{k \leftarrow \{0,1\}^\lambda} \left[\begin{array}{l} (\xi, \xi') \leftarrow \mathcal{A}(k) \text{ where:} \\ \alpha_k(\xi, \emptyset). \Delta = \alpha_k(\xi', \emptyset). \Delta \neq \perp \wedge \\ \bigcup \xi \neq \bigcup \xi' \end{array} \right] \in Negl(\lambda)$$

For completeness, we also define the (weaker) notion of a *target collision resistant (TCR)* accumulator. Here, the attacker has to select the ‘target’ sequence ξ_T and the state s before it is given the (random) key k . Only then, the attacker is given k and the output of the accumulator, using k , applied to ξ_T and state s , and should output the other sequence ξ' . The attacker ‘wins’ similarly to the conditions for ACR (above); the strings in ξ should differ from the strings in ξ' , yet their digests should be identical. The accumulator is TCR if this occur with (at most) negligible probability as a function of λ .

Definition 9 (Consistent Accumulator Target-Collision-Resistant). We say that the (keyed) consistent accumulator α is *Target-Collision-Resistant (TCR)* if for every efficient PPT algorithm $\mathcal{A}$ the following holds:

$$\varepsilon_{\alpha, \mathcal{A}}^{TCR}(\lambda) \equiv \Pr_{k \leftarrow \{0,1\}^\lambda} \left[\begin{array}{l} \left\{ (\xi_T, s_A) \leftarrow \mathcal{A}(1^\lambda) \right\} \\ \left\{ \xi \leftarrow \mathcal{A}(s_A, k) \right\}, \\ \text{where:} \\ \alpha_k(\xi_T, \emptyset). \Delta = \alpha_k(\xi, \emptyset). \Delta \neq \perp \\ \wedge \bigcup(\xi_T) \neq \bigcup(\xi) \end{array} \right] \in Negl(\lambda)$$

While we define collision resistance for completeness, proof of inclusion unforgeability guarantees collision resistance of the scheme.

Lemma 1 (PoI correctness plus unforgeability implies ACR). If a (keyed) consistent accumulator scheme α satisfies PoI correctness (Def. 2) and PoI unforgeability (Def. 5), then it satisfies ACR (Def. 8).

Proof. Assume to the contrary that α is not ACR, i.e., there exists a PPT adversary A_{ACR} such that with non-negligible probability over $k \leftarrow \{0,1\}^\lambda$, $A_{ACR}(k)$ outputs (ξ, ξ') such that

$$\Delta \equiv \alpha_k(\xi, \emptyset). \Delta = \alpha_k(\xi', \emptyset). \Delta \neq \perp$$

and

$$\bigcup(\xi) \neq \bigcup(\xi').$$

Without loss of generality, assume $\exists x \in \bigcup(\xi)$ s.t. $x \notin \bigcup(\xi')$.

We now build a PPT adversary A_{PoI} against Def. 5. On input k , adversary A_{PoI} runs $A_{ACR}(k)$ and obtains (ξ, ξ') as above. Then A_{PoI} computes

$$(s', \Delta, PoI, u, PoC) \leftarrow \alpha_k.Accum(\xi, \emptyset).$$

Since $x \in \bigcup(\xi)$, PoI correctness (Def. 2) implies that there exists $p \in PoI$ such that

$$\alpha_k.VerPol(\Delta, x, p) = \text{TRUE}.$$

Now observe that the same digest Δ is also obtained from ξ' , i.e.,

$$\Delta = \alpha_k(\xi', \emptyset). \Delta.$$

Recall that $x \notin \bigcup(\xi')$. Hence the pair (x, p) is a successful forgery against Def. 5: it verifies under a digest corresponding to an accumulation in which x was not accumulated. Therefore, $\varepsilon_{\alpha, A_{ACR}}^{ACR}(\lambda) = \varepsilon_{\alpha, A_{PoI}}^{PoI}(\lambda)$. $\square$

D. Extended Consistent Accumulator Schemes

Every consistent accumulator must provide the basic functionality in Def. 1: add elements, verify PoIs, verify updates, verify consistency, and update PoIs. This baseline is enough to capture correctness and security, and it keeps the abstraction clean. In practice, however, many applications benefit from helper procedures that avoid redundant work across extensions. The extended variant in Def. 10 formalizes this with the algorithms such as `GetU` and `GetPoC`, which let verifiers and monitors compute update values and consistency proofs directly, and combine multiple updates into a single object.

Definition 10 (Extended consistent accumulator schemes). An *extended consistent accumulator* extends Def. 1 with the following PPT algorithms:

$\alpha_k.GetU(ctr_0, ctr_1, s) \rightarrow u$: computes an update value that transforms PoIs valid after ctr_0 accumulations into PoIs valid after $ctr_1 > ctr_0$ accumulations, relative to the public state s .

$\alpha_k.GetPoC(ctr_0, ctr_1, s) \rightarrow PoC$: computes a proof of consistency between the digest after ctr_0 accumulations and the digest after $ctr_1 > ctr_0$ accumulations, relative to s .

E. Extended Consistent Accumulator Correctness Requirements:

Definition 11 (GetU correctness). We say that a consistent (keyed) accumulator α ensures GetU *correctness* if the following holds for every binary string key k , every initial binary string state s_0 , and any finite sequences of binary strings X and sequence of sequences ξ :

$$\left(\begin{array}{l} s_1 \leftarrow \alpha_k.\text{Accum}(X, s_0).s \wedge \\ \Delta_{ctr_0} \leftarrow \alpha_k.\text{Accum}(X, s_0).\Delta \wedge \\ s_2 \leftarrow \alpha_k.\text{Accum}(\xi, s_1).s \wedge \\ \Delta_{ctr_1} \leftarrow \alpha_k.\text{Accum}(\xi, s_1).\Delta \wedge \\ ctr_0 \leftarrow s_1.n \wedge ctr_1 \leftarrow s_2.n \wedge ctr_0 < ctr_1 \wedge \\ u \leftarrow \alpha_k.\text{GetU}(ctr_0, ctr_1, s_1) \end{array} \right) \Rightarrow \left(\begin{array}{l} \alpha_k.\text{VerUpd}(\Delta_{ctr_0}, u, \Delta_{ctr_1}) = \text{TRUE} \wedge \\ (\forall x, p : \alpha_k.\text{VerPol}(\Delta_{ctr_0}, x, p) = \text{TRUE} \wedge \\ p' \leftarrow \alpha_k.\text{UpdPol}(p, u) \\ \wedge \alpha_k.\text{VerPol}(\Delta_{ctr_1}, x, p') = \text{TRUE}) \end{array} \right)$$

Definition 12 (GetPoC correctness). We say that a consistent (keyed) accumulator α ensures GetPoC *correctness* if the following holds for every binary string key k , every initial binary string state s_0 , and any finite sequences of binary strings X and sequence of sequences ξ :

$$\left(\begin{array}{l} s_1 \leftarrow \alpha_k.\text{Accum}(X, s_0).s \wedge \\ \Delta_{ctr_0} \leftarrow \alpha_k.\text{Accum}(X, s_0).\Delta \wedge \\ s_2 \leftarrow \alpha_k.\text{Accum}(\xi, s_1).s \wedge \\ \Delta_{ctr_1} \leftarrow \alpha_k.\text{Accum}(\xi, s_1).\Delta \wedge \\ ctr_0 \leftarrow s_1.n \wedge ctr_1 \leftarrow s_2.n \wedge ctr_0 < ctr_1 \wedge \\ PoC \leftarrow \alpha_k.\text{GetPoC}(ctr_0, ctr_1, s_1) \end{array} \right) \Rightarrow (\alpha_k.\text{VerPoC}(\Delta_{ctr_0}, PoC, \Delta_{ctr_1}) = \text{TRUE}).$$

III. $\mathcal{CMT}$: EFFICIENT MERKLE-TREE CONSISTENT ACCUMULATOR

A. The $\mathcal{CMT}^h$ construction

In this section, we present $\mathcal{CMT}^h$, an efficient consistent accumulator construction based on the Merkle tree. Here, the superscript h denotes the collision-resistant hash function used in the construction. $\mathcal{CMT}^h$ maintains the same digest and PoI format as of a Merkle tree accumulator, for example, the Merkle tree implemented in the *Transparency.dev* library [2], also written in GoLang, used by Google's Certificate Transparency PKI system.

However, in contrast to *Transparency.dev*, $\mathcal{CMT}^h$ supports dynamic updates with compact recomputation, by caching the roots of previously computed subtrees and reusing them when possible. The result is an efficient accumulator whose PoI length and number of hashes required for verification is only $O(\log(n))$, where n is the (total) number of accumulated elements, and whose update operations require only $O(|X_{l+1}| + \log(n))$ hash operations when adding a new sequence X_{l+1} .

In our construction, the Proof of Consistency (*PoC*) and the update value (u) are identical, making them interchangeable. For example, the *UpdPol* function can accept either a u or a *PoC* as its input. Although conceptually distinct, this

identity means they share the same logic for generation (*Accum*, *GetPoC*, *GetU*) and verification algorithms (*VerPoC*, *VerUpd*).

$\mathcal{CMT}^h$ is an efficient consistent accumulator, whose state s is defined as a tuple of four components (n, DM, UM, ctr) :

- $s.n$: The total number of strings accumulated.
- $s.DM$: A map from an index pair (i, j) to the corresponding digest $\Delta_{i,j}$. We write entries as $(i, j) \mapsto \Delta_{i,j}$ and use $\Delta_{i,j} \leftarrow s.DM[(i, j)]$ for lookups.
- $s.ctr$: The number of accumulation events that have occurred.
- $s.UM$: A map used for efficient generation of update values, which is detailed in Alg. 5.
- $s.stateHist$: A list of references to previous states, used to retrieve the starting states for updates and PoCs.

In the event that we receive ϵ as the state, we initialize the state with $s.n = s.ctr = 0$ and $s.DM = s.UM = \emptyset$.

In the construction, the state s essentially acts as a cache, storing the root digests of all maximal full subtrees within the accumulator. This structure is the key to efficiency, as it allows for incremental updates without needing to re-access all historical elements. When new strings are added, $\mathcal{CMT}^h$ simply computes the new digests and updates the cache with the new set of maximal subtree roots.

For example, Fig. 1a shows the state after accumulating six elements $(x_0, \dots, x_5)$. The cache holds the two digests corresponding to the maximal subtrees: $\{\Delta_{0,4}, \Delta_{4,6}\}$. After adding $\{x_6, x_7\}$, the structure forms a single maximal full subtree, and the new state becomes just $\{\Delta_{0,8}\}$, as shown in Fig. 1b. In our implementation, this cache is populated by the $\text{CompCache}_{i,j}^s(X)$ algorithm. We now proceed with describing the pseudocode algorithms in the $\mathcal{CMT}^h$ design.

a) *Accumulating element sequences* (Alg. 1): The *Accum* algorithm uses four functions to compute its output: *CompUDM*, *CompPolAll*, *CompCache*, and *CompDigest*.

In every call to *Accum* from s_i to s_{i+1} , we update the global $s.UM$ with the nodes needed for update/PoC between s_i and s_{i+1} . We also hold the invariant that if there exists two sibling nodes in $s.UM$, their parent must also exist in $s.UM$. Holding this invariant ensures that *GetPoC* is efficient and does not need to compute new digests. Merging siblings is performed by *MergeUpdDig*, which computes the parent of sibling digests and recurses on the new parent.

b) *Computing digests* (Alg. 2): The *CompDigest* algorithm recursively computes the digest value of a given root node.

To compute the digest of a given range, *CompDigest* recursively hashes the concatenation of its left and right children's digests. It determines the split point between these children using the $lP_2(d)$ function, which finds the largest power of two less than or equal to the range size, d . This value, m , defines the midpoint between two child subtrees.

For example, to compute the root digest $\Delta_{0,6}$ in Fig. 1a, the algorithm first finds the midpoint $m = lP_2(6) = 4$. This splits the task into two sub-problems: computing the digest for the left subtree on indices $[0, 4)$, which yields $\Delta_{0,4}$, and the right

Algorithm 1 $\mathcal{CMT}_k^h.\text{Accum}(X, s) \rightarrow (s', \Delta, \text{PoI}, u, \text{PoC})$

```

1 : if  $s = \epsilon$ 
2 :    $s.n = 0$ 
3 :    $s.DM = \emptyset$ 
4 :    $s.UM = \emptyset$ 
5 :    $s.stateHist = \emptyset$ 
6 :    $s'.n = s.n + |X|$  new total number of elements accumulated
7 :    $u \leftarrow (s, \text{CompUDM}_{s'.n}^s(X), s'.n)$ 
8 :   if  $s.n > 0$ 
9 :      $s'.stateHist \leftarrow s.stateHist.append(\{s.n, s.DM\})$ 
10 :     $s'.UM = s.UM$ 
11 :    for each  $((i, j) \mapsto \Delta_u) \in u.DM$ 
12 :       $s.UM \leftarrow s.UM \cup \{(i, j) \mapsto \Delta_u\}$ 
13 :       $s.UM \leftarrow \text{MergeUpdDig}(s', (i, j) \mapsto \Delta_u)$ 
14 :      modifies UM to make it  $s_{temp}.DM = u.DM \cup s.DM$ 
15 :       $s_{temp}.n = s'.n$ 
16 :       $s'.DM \leftarrow \text{CompCache}_{0, s'.n}^{s_{temp}}(\emptyset)$ 
17 :      update state using old state plus  $u.DM$ 
18 :       $s'.\Delta \leftarrow \text{CompDigest}_{0, s'.n}^{s'}(\emptyset)$ 
19 :      compute root digest using elements in  $s'$ 
20 :       $\text{PoI} \leftarrow \text{CompPolAll}_{0, s.n}^s(X)$ 
21 :      Pol of new elements, for each new element in  $X$ 
22 :      return  $(s', s'.\Delta, \text{PoI}, u, u)$ 
23 :      PoC is the same as update value  $u$  in our construction.

```

subtree on indices $[4, 6)$, which yields $\Delta_{4,6}$. The final result, $\Delta_{0,6}$, is the hash of these two sub-trees' digests.

Algorithm 2 $\text{CompDigest}_{i,j}^s(X) \rightarrow \Delta$

```

1 : inputs should satisfy  $i < j \leq s.n$ 
2 : if  $\exists \Delta$  such that  $s.DM[(i, j)] = \Delta$ 
3 :   return  $\Delta$  digest ( $\Delta$ ) of element was previously computed
4 : else
5 :   if  $j - i = 1$ 
6 :     return  $h_k(0x00 || X[i - s.n])$  hash of single element
7 :   else
8 :      $m \leftarrow$  largest power of 2  $\leq j - i$ 
9 :     return  $h_k(0x01 || \text{CompDigest}_{i, i+m}^s(X) || \text{CompDigest}_{i+m, j}^s(X))$ 
10 :    digest of a sub-tree containing a left and right sub-trees

```

c) *Populating the state digests (Alg. 3):* For each batch added, CompCache computes $s'.DM$, the minimum set of left-aligned full subtrees that span every leaf in the new state s' . CompCache achieves this by recursively calling CompDigest on root nodes of smaller subtrees. We avoid recomputing digests at this point by first computing the update value u and using this value to update the old state by creating a temporary cache s_{temp} , which contains digests in the old state and update values computed from CompUDM.

d) *Computing update digests (Alg. 4):* CompUDM computes the dyadic cover, i.e., the minimal set of full subtrees with leaves that span the range from the old state to the new state. The set of root digests of all these ranges is computed with calls to CompDigest and used later in Accum to efficiently compute the new state's root as well as to update PoI in the future.

e) *Optimizing Update Storage (Alg. 5):* The function MergeUpdDig computes the update map $s.UM$ in a compact,

Algorithm 3 $\text{CompCache}_{i,j}^s(X) \rightarrow DM$

```

1 : if  $i \geq j$ 
2 :   return  $\emptyset$ 
3 : else
4 :    $m \leftarrow$  largest power of 2  $\leq j - i$ 
5 :   return  $\{(i, i+m) \mapsto \text{CompDigest}_{i, i+m}^s(X)\}$ 
6 :      $|| \text{CompCache}_{i+m, j}^s(X)$ 
7 :     add the digest of the  $2^{\log_2(m)}$  elements to the DM,
8 :     and contact the digests of the full trees from  $i+m$  to  $j-1$ 

```

Algorithm 4 $\text{CompUDM}_n^s(X) \rightarrow \{(i, j) \mapsto \Delta\}$

```

1 :  $DM \leftarrow \emptyset$ 
2 :  $i \leftarrow s.n$ 
3 : while  $i < n$ 
4 :    $m \leftarrow$  the largest power of 2 which divides  $i$ 
5 :   compute the largest possible subtree  $(i, i+m)$  with a leftmost leaf at  $i$ 
6 :   if  $m > n - i$ 
7 :      $m \leftarrow$  largest power of 2  $\leq j - i$ 
8 :   if we accumulated less than  $m$  items,
9 :   then compute largest perfect subtree that fits
10 :    $DM \leftarrow DM \cup \{(i, i+m) \mapsto \text{CompDigest}_{i, i+m}^s(X)\}$ 
11 :    $i \leftarrow i + m$ 
12 : return  $DM$ 

```

canonical form by merging together adjacent dyadic ranges into their parent node whenever both siblings are present. The parent is inserted into $s.UM$ and the procedure continues the recursion. If one sibling node is not present, the process stops. This guarantees the storage of maximal full subtrees.

Algorithm 5 $\text{MergeUpdDig}(s, (i, j) \mapsto \Delta)$

```

1 :  $span \leftarrow j - i$ 
2 : if  $i \equiv 0 \pmod{span * 2}$ 
3 :    $i_{sib} \leftarrow j$ 
4 :    $j_{sib} \leftarrow j + span$ 
5 : else
6 :    $j_{sib} \leftarrow i$ 
7 :    $i_{sib} \leftarrow i - span$ 
8 :   if  $\exists \Delta_{sib}$  s.t.  $s.UM[(i_{sib}, j_{sib})] = \Delta_{sib}$ 
9 :      $i_{prt} \leftarrow \min(i, j, i_{sib}, j_{sib})$ 
10 :     $j_{prt} \leftarrow \max(i, j, i_{sib}, j_{sib})$ 
11 :    if  $i_{sib} < i$ 
12 :       $\Delta_{prt} \leftarrow h_k(0x01 || \Delta_{sib} || \Delta)$ 
13 :    else
14 :       $\Delta_{prt} \leftarrow h_k(0x01 || \Delta || \Delta_{sib})$ 
15 :     $s.UM \leftarrow s.UM \cup \{(i_{prt}, j_{prt}) \mapsto \Delta_{prt}\}$ 
16 :     $\text{MergeUpdDig}(s, (i_{prt}, j_{prt}) \mapsto \Delta_{prt})$ 

```

f) *Retrieving updates and PoCs (Alg. 6):* When a client calls $\text{GetU}(\text{ctr}_0, \text{ctr}_1, s)$ or $\text{GetPoC}(\text{ctr}_0, \text{ctr}_1, s)$ (Alg. 6), they are requesting a joined update from the tree after ctr_0 calls of Accum to the tree after ctr_1 calls of Accum. This algorithm computes the locations of the dyadic cover nodes of the leaves between states ctr_0 and ctr_1 . Then it fetches the digests from $s.UM$, which must exist because of the procedure outlined in Accum and MergeUpdDig.

g) *Computing and updating PoIs (Algorithms 9 and 10):* The PoI is the proof of inclusion that verifies membership of a specific element in the tree. For example, in Fig. 1a, the PoI p of element x_1 will contain the following digests:

Algorithm 6 $\mathcal{CMT}_k^h.\text{GetPoC}(ctr_0, ctr_1, s) \rightarrow u$ and $\mathcal{CMT}_k^h.\text{GetU}(ctr_0, ctr_1, s) \rightarrow u$. Comment: In $\mathcal{CMT}$, both functions have the same implementation.

```

1 :  $s_i \leftarrow s.\text{stateHist}[ctr_0]$  // state corresponding to  $ctr_0$ 
2 :  $s_f.n \leftarrow s.\text{stateHist}[ctr_1]$  // # of elms. in state corresponding to  $ctr_1$ 
3 :  $u.s \leftarrow s_i$ 
4 :  $u.n \leftarrow s_f.n$ 
5 :  $u.DM \leftarrow \emptyset$ 
6 :  $i \leftarrow s_i.n$ 
7 : while  $i < s_f.n$ 
8 :    $m \leftarrow$  largest power of 2 which divides  $i$ 
9 :   if  $m = 0 \vee m > s_f.n - i$ 
10 :     $m \leftarrow$  largest power of 2  $\leq s_f.n - i$ 
11 :    $\Delta \leftarrow s.UM[(i, i + m)]$ 
12 :    $u.DM \leftarrow u.DM \cup \{(i, i + m) \mapsto \Delta\}$ 
13 :    $i \leftarrow i + m$ 
14 : return  $u$ 

```

$\{\Delta_{0,1}, \Delta_{2,4}, \Delta_{4,6}\}$. ComPolDM computes this set of digests for a particular element with index ID by recursively traversing down either the left or right side of the current subtree.

The old state's digests and new digest sets in our PoCs also serve as the update value u , which, in our construction, can be used to update an old PoI. Specifically, $u.DM$ contains exactly the additional full-subtree digests needed to construct the changed digests in a new PoI. Concretely, for x_1 , the new PoI is obtained by replacing the top sibling $\Delta_{4,6}$ with $\Delta_{4,8} = h(0x01 \parallel \Delta_{4,6} \parallel \Delta_{6,8})$, while keeping the lower siblings $\Delta_{0,1}$ and $\Delta_{2,4}$ unchanged.

Algorithm 7 $\text{CompPolState}_{i,j}^{s,s_{poi}}(X) \rightarrow \Delta$

```

1 : // inputs should satisfy  $i < j \leq s.n$ 
2 : if  $\exists \Delta$  such that  $s.DM[(i, j)] = \Delta$ 
3 :   // digest ( $\Delta$ ) of element was previously computed
4 : else if  $j - i = 1$ 
5 :    $\Delta \leftarrow h_k(0x00 \parallel X[i - s.n])$  // digest of single element
6 : else
7 :    $m \leftarrow$  largest power of 2  $\leq j - i$ 
8 :    $\Delta \leftarrow h_k(0x01 \parallel \text{CompDigest}_{i,i+m}^s(X)$ 
9 :      $\parallel \text{CompDigest}_{i+m,j}^s(X))$ 
10 :   // digest of a sub-tree containing a left and right sub-trees
11 :    $s_{poi}.DM \leftarrow s_{poi}.DM \cup \{(i, j) \mapsto \Delta\}$ 
12 : return  $\Delta$ 

```

Algorithm 8 $\text{CompPolAll}_{i,j}^s(X) \rightarrow PoIs$

```

1 : // inputs should satisfy  $0 \leq i < j$ 
2 :  $PoIs \leftarrow \emptyset$ 
3 :  $s_{poi}.n = s.n + |X|$ 
4 :  $s_{poi}.DM = s.DM$ 
5 :  $\text{CompPolState}_{i,j}^{s,s_{poi}}(X)$  // populate DM with all new digests
6 : for  $ID \in \{j, j + |X|\}$ 
7 :    $\text{CompPolDM}_{i,j}^{s_{poi}}(X)$ 
8 :   // compute every Pol using shared state to avoid recomputing new digests
9 : return  $PoIs$ 

```

h) Verifying PoI, PoC, and updates (Algorithms 11 and 12): Both verification algorithms reconstruct the root digest of the latest tree. Verification succeeds if the reconstructed root digest matches the one passed by the client; otherwise, it fails.

Algorithm 9 $\text{ComPolDM}[ID]_{i,j}^s(X) \rightarrow DM$

```

1 : // inputs should satisfy  $0 \leq i < j$ 
2 :  $m \leftarrow$  largest power of 2  $< j - i$ 
3 : if  $ID < m$  // target is in left subtree; include right sibling
4 :   return  $(i + m, j, \text{CompDigest}_{i+m,j}^s(X)) \cup$ 
5 :      $\text{ComPolDM}[ID]_{i,i+m}^s(X)$ 
6 : else // target is in right subtree; include left sibling
7 :   return  $(i, i + m, \text{CompDigest}_{i,i+m}^s(X)) \cup$ 
8 :      $\text{ComPolDM}[ID - m]_{i+m,j}^s(X)$ 

```

Algorithm 10 $\mathcal{CMT}_k^h.\text{UpdPol}(p, u) \rightarrow p'$

```

1 : //  $u.n$  is num elements after update
2 :  $s'.n \leftarrow u.n$  // update length
3 :  $s'.DM \leftarrow u.s.DM \cup p.DM \cup u.DM$ 
4 : //  $u.s.DM, p, u.DM$  provides all necessary values to compute new Pol
5 :  $p'.i \leftarrow p.i$ 
6 :  $p'.n \leftarrow s'.n$ 
7 :  $p'.DM \leftarrow (\text{ComPolDM}[p.i]_{0,s'.n}^{s'}(\emptyset))$ 
8 : //  $p$  includes new vals for PoI
9 : return  $p'$ 

```

VerPol reconstructs the root starting from the leaf by recursively hashing the current digest with its sibling (the next-highest digest in the PoI digest set).

Alg. 12, which implements both VerPoC and VerUpd, uses CompDigest to compute the old and new roots using the old state $PoC.s/u.s$. The old state's digests combined with the digests in $PoC.DM/u.DM$ are enough to construct the new root.

Algorithm 11 $\mathcal{CMT}_k^h.\text{VerPol}(\Delta, x, p) \rightarrow \{\text{TRUE or FALSE}\}$

```

1 : return  $\Delta = \text{PolDigest}(x, p)$ 
2 :
3 :  $\text{PolDigest}(x, p) \rightarrow \Delta \{$ 
4 :   if  $p.n \leq 1$  then return  $h_k(0x00 \parallel x)$ 
5 :   if  $|\pi.DM| = 0$  then return  $\perp$ 
6 :    $((i, j) \mapsto \Delta') \leftarrow$  find map entry with the smallest  $j - i$  value
7 :     in  $p.DM$ 
8 :    $p'.DM \leftarrow p.DM \setminus \{(i, j) \mapsto \Delta'\}$ 
9 :    $m \leftarrow lP_2(p.n - 1)$ 
10 :  if  $p.i < n$  // PoI target is in the left subtree
11 :     $p'.n \leftarrow n$ 
12 :     $p'.i \leftarrow p.i$ 
13 :    return  $h_k(0x01 \parallel \text{PolDigest}(x, p') \parallel \Delta')$ 
14 :  else // PoI target is in the right subtree
15 :     $p'.n \leftarrow p.n - n$ 
16 :     $p'.i \leftarrow p.i - n$ 
17 :    return  $h_k(0x01 \parallel \Delta' \parallel \text{PolDigest}(x, p'))$ 
18 : }

```

Algorithm 12 Algorithm of $\mathcal{CM}\mathcal{T}_k^h.\text{VerPoC}(\Delta, \text{PoC}, \Delta')$ and $\mathcal{CM}\mathcal{T}_k^h.\text{VerUpd}(\Delta, u, \Delta')$ where both return either TRUE or FALSE.

```

1:  $\text{VerPoC}(\Delta, \text{PoC}, \Delta') \rightarrow \{\text{TRUE or FALSE}\}\{$ 
2:    $\text{return VerUpdnPoC}(\Delta, \text{PoC}, \Delta')$ 
3:  $\}$ 
4:  $\text{VerUpd}(\Delta, u, \Delta') \rightarrow \{\text{TRUE or FALSE}\}\{$ 
5:    $\text{return VerUpdnPoC}(\Delta, u, \Delta')$ 
6:  $\}$ 
7:  $\text{VerUpdnPoC}(\Delta, o, \Delta') \rightarrow \{\text{TRUE or FALSE}\}\{$ 
8:    $\text{// Here } o \text{ is the tuple of } (s, DM, n) \text{ representing either } \text{PoC} \text{ or update value}$ 
9:    $s'.n \leftarrow o.n$ 
10:   $s'.DM \leftarrow o.s.DM \cup o.DM$ 
11:   $\text{// } \Delta' \text{'s covering range of all elements accumulated}$ 
12:   $\text{if CompDigest}_{0,s'.n}^{o.s}(\emptyset) \equiv \Delta \text{ and}$ 
13:     $\text{CompDigest}_{0,s'.n}^{s'}(\emptyset) \equiv \Delta'$ 
14:     $\text{return True}$   $\text{// original } \Delta \text{ is consistent with original state and}$ 
15:     $\text{// new } \Delta' \text{ is consistent with PoC values}$ 
16:   $\text{else return False}$ 
17:  $\}$ 
```

IV. EVALUATION

We implement our $\mathcal{CM}\mathcal{T}^h$ design in Go, enabling direct comparison with Google's open-source *Transparency.dev* implementation [2], which is written in Go and deployed in many production CT-like logs. This provides a realistic benchmark against an industry-standard CT codebase under the same language and runtime environment.

All experiments were executed using an AMD Ryzen 9 7950X3D desktop processor, one CPU core, 64 GiB memory, AMD64 Linux, and Go version 1.25.4. Each leaf hash in our experiments is a SHA-256 digest of 100 randomly generated bytes, using APIs from the crypto/sha256 package. For each experiment, we integrate $\mathcal{CM}\mathcal{T}^h$ and Google's implementation into the same benchmarking harness to ensure identical conditions, isolating the performance differences attributable to our accumulator design. Values were chosen that span multiple orders of magnitude for algorithms that exhibited logarithmic complexity. Further experimentation showed that values that are exact powers of 2 and near powers of 2 exhibit very low and high memory usage, respectively. A similar pattern was noticed for runtime. Therefore, as a compromise, we chose values as $1.5 \cdot 2^k, k \in \mathbb{N}$, i.e. halfway between powers of 2. For each runtime data point, we express the mean with error bars to denote the 95% confidence interval across 50 runs. Because our memory values do not fluctuate across runs, each memory data point is the value of a single run. We made efforts to limit background processes in user space while running tests, but some figures consistently fail to exhibit clear monotonic trends. We hypothesize the reason is a combination of the Go garbage collector, dynamic allocators, and memory locality.

a) *Accumulation Performance*: Fig. 3 and Fig. 4 show the time and space required to:

- 1) Add elements to an initially empty log (starting with 0 elements).
- 2) Append an additional batch of elements to an existing log (incremental update).

While Google's implementation stores all digests of the current tree, recomputing internal nodes as required for each addition, $\mathcal{CM}\mathcal{T}^h$ uses cached maximal subtrees to avoid recomputation for unchanged portions of the tree. As seen in Fig. 3, the memory allocation of $\mathcal{CM}\mathcal{T}^h$ significantly outperforms that of *Transparency.dev*. The second experiment measuring the time it takes to accumulate elements, shown in Fig. 4, demonstrates how incremental accumulation with caching improves throughput in large CT logs. We can see that the storage of a large state as in *Transparency.dev* negatively influences performance, causing the addition time to vary widely between accumulation events.

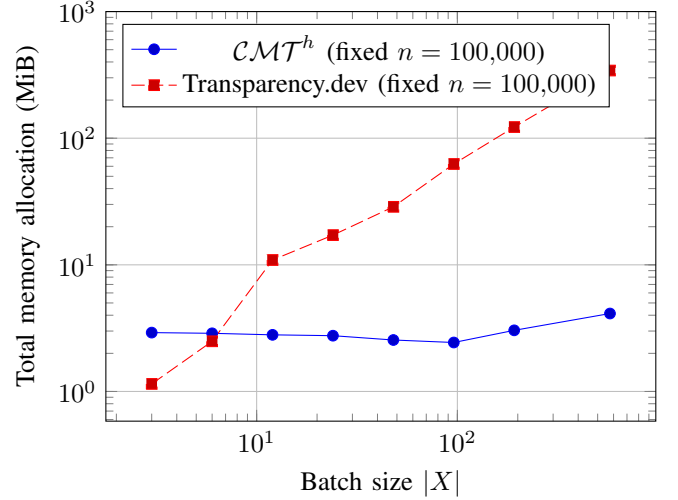


Fig. 3: Appending various batch sizes $|X|$ to a log of fixed size $n = 100,000$. For each experiment, the log has a starting size of 100,000 elements. We measure the total memory allocated as a result of appending 1000 batches, each of size $|X|$.

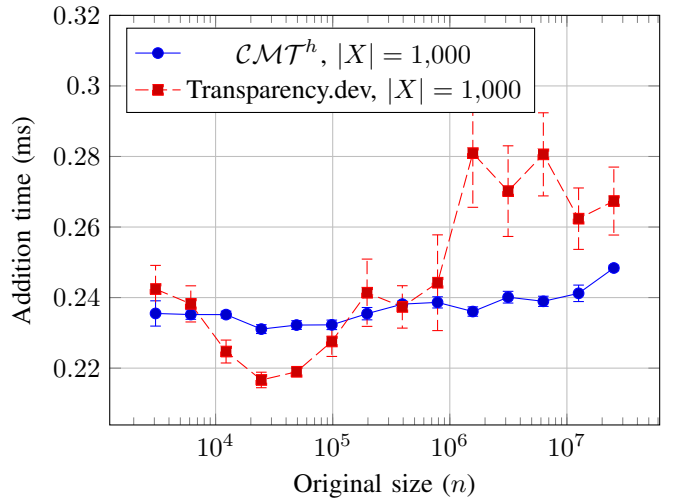


Fig. 4: Incremental append of a fixed batch $|X| = 1,000$ as n grows. In each experiment, a varied n elements are added first and we measure the time it takes to add 1,000 elements.

In Fig. 4 we show accumulation time vs. number of elements,

comparing $\mathcal{CMT}^h$ to Google's *Transparency.dev* with fixed $|X| = 1,000$ as n increases (we also measured varying $|X|$ at fixed $n = 100,000$, and do not present these results as they showed similar results).

b) *Proof-of-Consistency and Update Verification Performance*: We do not present PoC and update verification performance as both $\mathcal{CMT}^h$ and *Transparency.dev* compute the same PoC from pre-generated digests, just returning the PoC from stored metadata. Note that $\mathcal{CMT}^h$'s reliance on batch accumulations means only PoCs between batch boundaries are supported. The advantage is needing to store less digests while still achieving the same $O(\log m)$ generation time. We do not present update verification performance data as updates are not supported by *Transparency.dev*.

c) *Proof-of-Consistency Generation*: We now measure the time required to generate a proof of consistency between two tree sizes. We ran experiments varying both:

- 1) The size n of the original tree.
- 2) The size of each element batch, $|X|$.

Due to the graphs showing the same trend, we only present here Fig. 5, varying tree size, but the results from varying batch size were similar, with $\mathcal{CMT}^h$ showing a similar but faster trend compared to *Transparency.dev*.

For each scenario, we begin with a log of size n and then append 1000 batches of size $|X|$. After every batch b_i is added, $i \in [1, 1000]$, let t_i be the new state of the log. When all 1000 batches are added, the final log t_{1000} has size $n + 1000 \cdot |X|$ elements. We measure the time it takes to generate 1000 proofs of consistency, from a random starting state $t_r, r \in \text{Uniform}\{1, \dots, 999\}$, to t_{1000} .

Since these tests compute significantly more per data point, we run the tests 7 times per point instead of 50.

Google's *Transparency.dev* generates PoCs after accumulation as defined in the CT standard [25]. $\mathcal{CMT}^h$ stores only the digests needed to generate a PoC between any two batch boundaries, i.e. states that result from calling Accum on a batch of elements. As a result, $\mathcal{CMT}^h$ saves space by storing only the necessary digests need to compute specific PoCs, but losing the ability to generate the fine-grained proofs to and from within batch appends. This tradeoff was described in more detail in Section I-A.

Fig. 5: Consistency proof generation time vs. number of elements, comparing $\mathcal{CMT}^h$ to Google's *Transparency.dev* varying $|X|$ at fixed $n = 100,000$ (we do not present, but we also measured fixed $|X| = 1,000$ as n increases, seeing similar results).

d) *Proof-of-Inclusion Generation*: We do not compare PoI generation time, as the implementations are fundamentally different. In contrast to *Transparency.dev* where PoIs are generated on demand when requested, we don't generate PoIs retroactively in $\mathcal{CMT}^h$, instead generating them during an accumulation event. In real systems objects and their corresponding PoIs are stored together. In a CT-like system, the PoI is sent with the certificate; in blockchains, the PoI should again be kept with the transaction. Retroactively requesting the log or block producer for a PoI is backwards (and expensive).

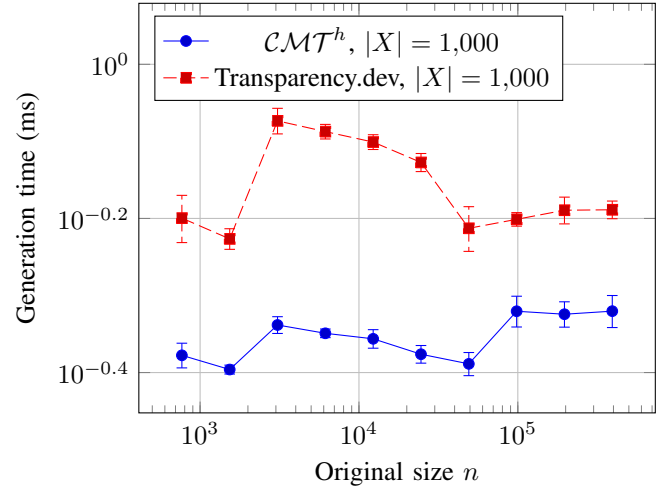


Fig. 5: Time to generate a PoC from 1000 random starting states to the most recent state. There are 1000 possible starting states, each resulting from appending a constant 1000 elements to the previous state. The earliest of these states has size n , which we vary in this test.

The current CT practice of retroactive PoI generation creates a huge performance tax; our design avoids that by producing updateable PoIs upfront, which matches the real-world use case for these systems.

A. Summary of Results

Across all experiments, $\mathcal{CMT}^h$ consistently outperforms Google's *Transparency.dev* implementation. In the CT setting, this translates directly into reduced CPU load on log servers, faster proof generation, and lower latency for monitors and auditors when synchronizing with the log.

V. ANALYSIS OF $\mathcal{CMT}^h$

In this section we analyze $\mathcal{CMT}^h$, proving that $\mathcal{CMT}^h$ satisfies the consistent accumulator requirements presented in Section II-A and Section II-B and the extended consistent accumulator requirements presented in Section II-D.

a) *Correctness requirements*: We first note that $\mathcal{CMT}^h$ ensures, unconditionally, the correctness requirements.

Lemma 2. $\mathcal{CMT}^h$ ensures the consistent accumulator correctness requirements: *PoI correctness*, *updating PoI correctness*, and *PoC and Update value correctness* (Defs. 2, 3, 4).

Proof. (1) **PoI Correctness.** By construction, each call to Accum replaces the state's digest map with the dyadic cover of $[0, s'.n)$ and sets $\Delta' = \text{CompDigest}_{0, s'.n}^s(\emptyset)$. Then the recomputed root from the full subtrees in $s'.DM$ is exactly the returned digest, giving PoI correctness for all newly added leaves, as their CompPoIDM paths are assembled from these subtrees.

(2) **Updating PoI Correctness.** For updating PoIs, $u = (s, u.DM, s'.n)$ contains precisely the full subtrees that cover the appended range $[s.n, s'.n)$; UpdPoI only swaps siblings that cross into this new range and leaves the original path before

the update the same. Hence the updated proof recomputes to $\text{CompDigest}_{\tilde{s}, s'.n}(\emptyset)$ where $\tilde{s}.DM = s.DM \cup u.DM$, which is computationally equivalent to $s'.DM$, so verification under Δ' holds.

(3) **PoC and Update value Correctness.** Finally, VerUpd and VerPoC each reconstruct a state from $(s, u.DM, s'.n)$ and check that the roots before and after match Δ and Δ' . Since the dyadic cover is maximal and aligned, the reconstructed map yields the same root as $s'.DM$, hence both verifiers accept. $\square$

b) Unforgeability requirements: Next, we show that $\mathcal{CM}T^h$ ensures the three unforgeability requirements, *PoI Unforgeability*, *update unforgeability*, and *PoC unforgeability*. *PoI (proof of inclusion) unforgeability* means that an adversary who sees any number of digests, states, and update values still cannot produce a valid PoI for an element that was never accumulated except with negligible probability. *Update unforgeability* means that once a PoI is valid, applying any valid update value preserves validity. *PoC (proof of consistency) unforgeability* means that the adversary cannot produce a valid PoC showing that a new digest Δ_1 is consistent with a previous digest Δ_0 , if there is some string included in Δ_0 but not in Δ_1 except with negligible probability.

Lemma 3. If the hash used by $\mathcal{CM}T^h$ is collision resistant, then $\mathcal{CM}T^h$ satisfies *PoI unforgeability*, *update unforgeability*, and *PoC unforgeability* (Defs. 5, 6, 7).

Proof. All three properties reduce to the fact that $\mathcal{CM}T^h$'s verifiers (VerPol , VerUpd , VerPoC) accept if and only if the recomputed root equals the claimed digest, where roots are computed by concatenating child digests over the dyadic cover.

(1) **PoI unforgeability.** A forged PoI for an element $x \notin \bigcup(\xi)$ means that there are two different leaf assignments that hash to the same root, which is a collision in the Merkle hashing.

(2) **Update unforgeability.** If a valid update u could turn a valid PoI (x, π) under Δ_0 into an invalid PoI under Δ_1 , or turn an invalid PoI into a valid one, then at some point in the tree hashing, a sibling node given (or omitted) by u would have to change the recomputed root without changing any of the siblings from the pre-update range. That would mean that two inputs map to the same node digest, i.e., a hash collision.

(3) **PoC unforgeability.** If an adversary produced a PoC accepted between Δ_0 and Δ_1 where Δ_1 is not an append-only extension, the verifier's reconstructed state would necessarily contain a subtree whose leaf contents do not match between the two histories yet yield the same root digest. This again implies a hash collision.

In all cases, collision resistance of the hashing function rules out these events except with negligible probability. $\square$

A. The $\mathcal{CM}T^h$ is collision resistant

Collision resistance for a consistent accumulator is analogous to collision resistance for static accumulators: no efficient adversary can produce two different element sets that yield the same digest. In $\mathcal{CM}T^h$, regardless of if elements are added all

at once or in batches, digests are Merkle roots computed over a fixed ordering of leaves. The cache only keeps subtrees of previously added elements and its use does not change the root function. Hence, under a collision-resistant hash, distinct sets should produce distinct roots, which gives ACR and therefore TCR.

Lemma 4. Assume the underlying hash function used by $\mathcal{CM}T^h$ is collision resistant. Then $\mathcal{CM}T^h$ is an ACR (Definition 8) and TCR (Definition 9) consistent accumulator.

Proof. Follows directly from Lemma 1. $\square$

Lemma 5. $\mathcal{CM}T^h$ satisfies extended accumulator correctness properties (Defs. 11 and 12).

Proof. (1) **GetU correctness.** Given $ctr_0 < ctr_1$, $\text{GetU}(ctr_0, ctr_1, s_1)$ returns $u = (s_1, u.DM, ctr_1)$ where $u.DM$ is the dyadic cover of $[ctr_0, ctr_1)$; $s_1.DM$ already covers $[0, ctr_0)$. The verifier forms $\tilde{s}$ with $\tilde{s}.DM = s_1.DM \cup u.DM$ and $\tilde{s}.n = ctr_1$. This map is *computationally equivalent* to the real map we get after the update, therefore $\text{CompDigest}_{\tilde{s}, ctr_1}(\emptyset) = \Delta_{ctr_1}$ and $\text{VerUpd}(\Delta_{ctr_0}, u, \Delta_{ctr_1}) = \text{TRUE}$. For any PoI π valid under Δ_{ctr_0} , $\text{UpdPol}(\pi, u)$ preserves siblings in $[0, ctr_0)$ and replaces only those crossing into $[ctr_0, ctr_1)$ from $u.DM$; the recomputed root is Δ_{ctr_1} , so verification holds.

(2) **GetPoC correctness.** $\text{GetPoC}(ctr_0, ctr_1, s_1)$ returns $PoC = (s_1, u.DM, ctr_1)$ with $u.DM$ the dyadic cover of $[ctr_0, ctr_1)$. The verifier reconstructs $\tilde{s}$ as above and checks the first root equals Δ_{ctr_0} (by definition) and the second root equals Δ_{ctr_1} . Since $\tilde{s}.DM = s_1.DM \cup u.DM$ is computationally equivalent to the honest state's map over $[0, ctr_1)$, the post-root matches and $\text{VerPoC}(\Delta_{ctr_0}, PoC, \Delta_{ctr_1}) = \text{TRUE}$. $\square$

VI. CONCLUSION

In this work, we introduced a precise definition for consistent accumulators, specifying a uniform interface and the exact security properties that a consistent accumulator should satisfy.

Building on this framework, we presented the Efficient Merkle-tree Consistent Accumulator ($\mathcal{CM}T^h$), a new Merkle-compatible consistent accumulator that preserves the standard digest and proof formats while enabling efficient dynamic updates. By caching the roots of maximal power-of-two subtrees, $\mathcal{CM}T^h$ performs incremental digest computation and PoI updates without ever re-hashing the entire dataset.

We proved that $\mathcal{CM}T^h$ meets the required security and consistency properties, and our run results as compared to *Transparency.dev* showed significant improvements as compared to the widely used CT standard.

Since $\mathcal{CM}T^h$ is a drop-in replacement for existing Merkle-based logs (e.g. Certificate Transparency) yet improves both on computation and storage overheads, it offers a practical upgrade path for systems that rely on append-only transparency logs, secure audit trails, or other authenticated data structures demanding scalability.

ACKNOWLEDGEMENTS

Ivan Pryvalov was supported by the Luxembourg National Research Fund (FNR), under the CORE Junior project FP2 (C20/IS/14698166/FP2/Mueller) and by the German Federal Government, the Federal Ministry of Education and Research and the State of Brandenburg within the framework of the joint project EIZ: Energy Innovation Center (project numbers 85056897 and 03SF0693A) with funds from the Structural Development Act (Strukturstärkungsgesetz) for coal-mining regions.

REFERENCES

- [1] B. Laurie, E. Messeri, and R. Stradling, “Certificate Transparency Version 2.0,” RFC 9162 (Experimental), RFC Editor, Fremont, CA, USA, Dec. 2021. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9162.txt>
- [2] Google, “Transparency.dev: Trust your data with a tamper-evident log,” <https://transparency.dev/>, 2025.
- [3] B. Laurie, “Certificate transparency,” *Communications of the ACM*, vol. 57, no. 10, pp. 40–46, 2014.
- [4] anonymous, “Golang implementation of cmt,” 2025. [Online]. Available: <https://anonymous.4open.science/r/optimized-dynamic-mt-go-2E2B/>
- [5] R. C. Merkle, “A Digital Signature Based on a Conventional Encryption Function,” in *Conference on the theory and application of cryptographic techniques*. Springer, 1987, pp. 369–378.
- [6] J. Benaloh and M. De Mare, “One-way accumulators: A decentralized alternative to digital signatures,” in *Workshop on the Theory and Application of Cryptographic Techniques*. Springer, 1993, pp. 274–285.
- [7] B. Laurie, A. Aas, D. Belcher, A. C. DeCarlo, M. Engert, E. Kasper, E. Messeri, M. Seigler, and R. Sleevi, “Trillian: General transparency,” in *2019 IEEE Cybersecurity Development (SecDev)*. IEEE, 2019, pp. 1–7.
- [8] J. Camenisch and A. Lysyanskaya, “Dynamic accumulators and application to efficient revocation of anonymous credentials,” in *Annual international cryptography conference*. Springer, 2002, pp. 61–76.
- [9] J. Li, N. Li, and R. Xue, “Universal accumulators with efficient nonmembership proofs,” in *International Conference on Applied Cryptography and Network Security*. Springer, 2007, pp. 253–269.
- [10] P. Camacho, A. Hevia, M. Kiwi, and R. Opazo, “Strong accumulators from collision-resistant hashing,” in *International Conference on Information Security*. Springer, 2008, pp. 471–486.
- [11] M. H. Au, P. P. Tsang, W. Susilo, and Y. Mu, “Dynamic universal accumulators for ddh groups and their application to attribute-based anonymous credential systems,” in *Cryptographers’ track at the RSA conference*. Springer, 2009, pp. 295–308.
- [12] H. de Meer, M. Liedel, H. C. Pöhls, J. Posegga, and K. Samelin, “Indistinguishability of one-way accumulators,” *Faculty Comput. Sci. Math., Univ. Passau, Passau, Germany, Tech. Rep. MIP-1210*, 2012.
- [13] D. Derler, C. Hanser, and D. Slamanig, “Revisiting cryptographic accumulators, additional properties and relations to other primitives,” in *Cryptographers’ track at the rsa conference*. Springer, 2015, pp. 127–144.
- [14] E. Ghosh, O. Ohrimenko, D. Papadopoulos, R. Tamassia, and N. Triandopoulos, “Zero-knowledge accumulators and set algebra,” in *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 2016, pp. 67–100.
- [15] F. Baldimtsi, J. Camenisch, M. Dubovitskaya, A. Lysyanskaya, L. Reyzin, K. Samelin, and S. Yakubov, “Accumulators with applications to anonymity-preserving revocation,” in *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2017, pp. 301–315.
- [16] S. A. Crosby and D. S. Wallach, “Efficient data structures for tamper-evident logging,” in *USENIX Security Symposium*, 2009, pp. 317–334.
- [17] B. Dowling, F. Günther, U. Herath, and D. Stebila, “Secure Logging Schemes and Certificate Transparency,” in *European Symposium on Research in Computer Security*. Springer, 2016, pp. 140–158.
- [18] D. Kales, O. Omolola, and S. Ramacher, “Revisiting user privacy for certificate transparency,” in *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2019, pp. 432–447.
- [19] A. Tomescu, V. Bhupatiraju, D. Papadopoulos, C. Papamanthou, N. Triandopoulos, and S. Devadas, “Transparency logs via append-only authenticated dictionaries,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019, pp. 1299–1316.
- [20] M. S. Melara, A. Blankstein, J. Bonneau, E. W. Felten, and M. J. Freedman, “CONIKS: Bringing Key Transparency to End Users,” in *USENIX Security Symposium*, 2015, pp. 383–398.
- [21] Y. Hu, K. Hooshmand, H. Kalidhindi, S. J. Yang, and R. A. Popa, “Merkle 2: A low-latency transparency log system,” in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 285–303.
- [22] D. Reijnders, A. Maw, Z. Yang, T. T. A. Dinh, and J. Zhou, “{TAP}: Transparent and {Privacy-Preserving} data services,” in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 6489–6506.
- [23] J. Len, M. Chase, E. Ghosh, K. Laine, and R. C. Moreno, “{OPTIKS}: An optimized key transparency system,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 4355–4372.
- [24] L. Reyzin and S. Yakubov, “Efficient asynchronous accumulators for distributed pki,” in *International Conference on Security and Cryptography for Networks*. Springer, 2016, pp. 292–309.
- [25] B. Laurie, A. Langley, and E. Kasper, “Certificate Transparency,” RFC 6962 (Experimental), RFC Editor, Fremont, CA, USA, Jun. 2013, obsoleted by RFC 9162. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc6962.txt>
- [26] P. Todd, “Merkle mountain ranges,” <https://github.com/opentimestamps/opentimestamps-server/blob/master/doc/merkle-mountain-range.md>, March 2013, openTimestamps documentation.
- [27] B. Bailey and S. Sankagiri, “Merkle trees optimized for stateless clients in bitcoin,” in *Financial Cryptography and Data Security. FC 2021 International Workshops: CoDecFin, DeFi, VOTING, and WTSC, Virtual Event, March 5, 2021, Revised Selected Papers 25*. Springer, 2021, pp. 451–466.
- [28] C. Papamanthou, S. Srinivasan, N. Gailly, I. Hishon-Rezaizadeh, A. Salumets, and S. Golemac, “Reckle trees: Updatable merkle batch proofs with applications,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 1538–1551.
- [29] A. Kate, G. M. Zaverucha, and I. Goldberg, “Constant-size commitments to polynomials and their applications,” in *International conference on the theory and application of cryptology and information security*. Springer, 2010, pp. 177–194.
- [30] D. Catalano and D. Fiore, “Vector commitments and their applications,” in *International Workshop on Public Key Cryptography*. Springer, 2013, pp. 55–72.
- [31] S. Srinivasan, A. Chepurnoy, C. Papamanthou, A. Tomescu, and Y. Zhang, “Hyperproofs: Aggregating and maintaining proofs in vector commitments,” in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3001–3018.
- [32] M. Yu, S. Sahraei, S. Li, S. Avestimehr, S. Kannan, and P. Viswanath, “Coded merkle tree: Solving data availability attacks in blockchains,” in *International Conference on Financial Cryptography and Data Security*. Springer, 2020, pp. 114–134.
- [33] M. Bellare and P. Rogaway, “Collision-resistant hashing: Towards making UOWHFs practical,” in *Annual International Cryptology Conference*. Springer, 1997, pp. 470–484.